

第四章 组合逻辑电路设计(二)

Combinational-Logic Circuits Design

秦磊华 汪承宁

4.6 基本组合逻辑功能部件设计

5. 编码器 (encoder)

1) 编码的基本概念

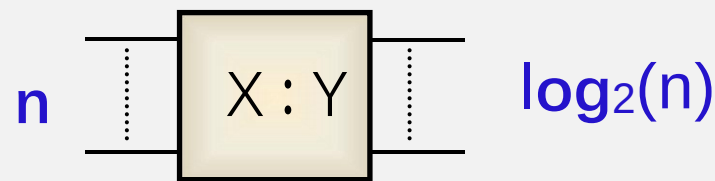
将文字、数字、符号等对象用二进制数字串表示。

2) 编码器

能够完成编码功能的电路叫编码器

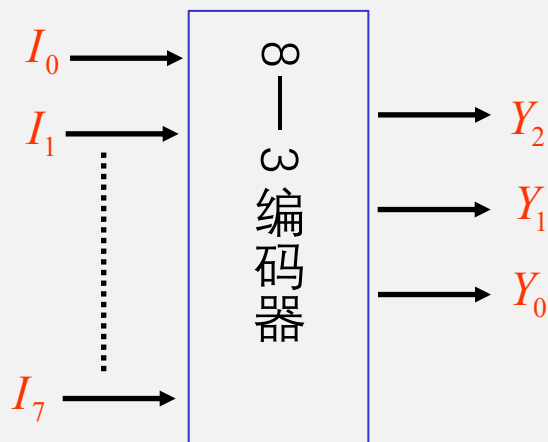
3) 编码器分类

普通编码器和优先级编码器



4.6 基本组合逻辑功能部件设计

4) 普通编码器



8-3编码器:

I_7-I_0 : 8个输入端, 高电平有效

$Y_2Y_1Y_0$: 是3位二进制代码输出

不允许同时输入多个编码信号

4.6 基本组合逻辑功能部件设计

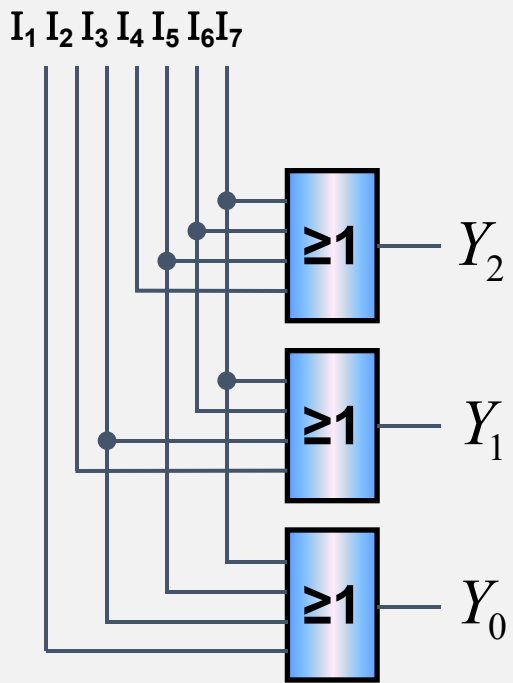
4) 普通编码器的真值表 (3张合并)

输 入								输 出		
I ₀	I ₁	I ₂	I ₃	I ₄	I ₅	I ₆	I ₇	Y ₂	Y ₁	Y ₀
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

$$Y_2 = I_4 + I_5 + I_6 + I_7$$

$$Y_1 = I_2 + I_3 + I_6 + I_7$$

$$Y_0 = I_1 + I_3 + I_5 + I_7$$



考察每一位输出，把0替换成don't care进行化简
这会导致如果输入非法，输出会是乱码（不是全0）

4.6 基本组合逻辑功能部件设计

真值表（3张合并）

5) 优先级编码器（priority encoder）

优先级编码器允许同时有几个输入端有输入信号，只对优先级最高的那个输入的位置进行编码。

I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	Y_2	Y_1	Y_0
1	X							1	1	1
0	1	X						1	1	0
0	0	1	X					1	0	1
0	0	0	1	X				1	0	0
0	0	0	0	1	X			0	1	1
0	0	0	0	0	1	X		0	1	0
0	0	0	0	0	0	1	X	0	0	1
0	0	0	0	0	0	0	1	0	0	0

$$Y_2 = I_7 + \overline{I_7} \cdot I_6 + \overline{I_7} \cdot \overline{I_6} \cdot I_5 + \overline{I_7} \cdot \overline{I_6} \cdot \overline{I_5} \cdot I_4 = I_7 + I_6 + I_5 + I_4$$

$$Y_1 = I_7 + \overline{I_7} \cdot I_6 + \overline{I_7} \cdot \overline{I_6} \cdot \overline{I_5} \cdot \overline{I_4} \cdot I_3 + \overline{I_7} \cdot \overline{I_6} \cdot \overline{I_5} \cdot \overline{I_4} \cdot \overline{I_3} \cdot I_2 = I_7 + I_6 + \overline{I_5} \cdot \overline{I_4} \cdot (I_3 + I_2)$$

$$Y_0 = I_7 + \overline{I_7} \cdot \overline{I_6} \cdot I_5 + \overline{I_7} \cdot \overline{I_6} \cdot \overline{I_5} \cdot \overline{I_4} \cdot I_3 + \overline{I_7} \cdot \overline{I_6} \cdot \overline{I_5} \cdot \overline{I_4} \cdot \overline{I_3} \cdot \overline{I_2} \cdot I_1 = I_7 + \overline{I_6} \cdot I_5 + \overline{I_6} \cdot \overline{I_4} \cdot I_3 + \overline{I_6} \cdot \overline{I_4} \cdot \overline{I_2} \cdot I_1$$

4.6 基本组合逻辑功能部件设计

$$Y_2 = I_4 + I_5 + I_6 + I_7$$

$$Y_1 = I_2 + I_3 + I_6 + I_7$$

$$Y_0 = I_1 + I_3 + I_5 + I_7$$

普通编码器

$$Y_2 = I_7 + \overline{I_7} \cdot I_6 + \overline{I_7} \cdot \overline{I_6} \cdot I_5 + \overline{I_7} \cdot \overline{I_6} \cdot \overline{I_5} \cdot I_4 = I_7 + I_6 + I_5 + I_4$$

$$Y_1 = I_7 + \overline{I_7} \cdot I_6 + \overline{I_7} \cdot \overline{I_6} \cdot \overline{I_5} \cdot \overline{I_4} \cdot I_3 + \overline{I_7} \cdot \overline{I_6} \cdot \overline{I_5} \cdot \overline{I_4} \cdot \overline{I_3} \cdot I_2 = I_7 + I_6 + \overline{I_5} \cdot \overline{I_4} \cdot (I_3 + I_2)$$

$$Y_0 = I_7 + \overline{I_7} \cdot \overline{I_6} \cdot I_5 + \overline{I_7} \cdot \overline{I_6} \cdot \overline{I_5} \cdot \overline{I_4} \cdot I_3 + \overline{I_7} \cdot \overline{I_6} \cdot \overline{I_5} \cdot \overline{I_4} \cdot \overline{I_3} \cdot \overline{I_2} \cdot I_1 = I_7 + \overline{I_6} \cdot I_5 + \overline{I_6} \cdot \overline{I_4} \cdot I_3 + \overline{I_6} \cdot \overline{I_4} \cdot \overline{I_2} \cdot I_1$$

优先级编码器

6.译码器 (decoder)

1)基本概念

编码器的逆，将输入的二进制码翻译成对应的输出。

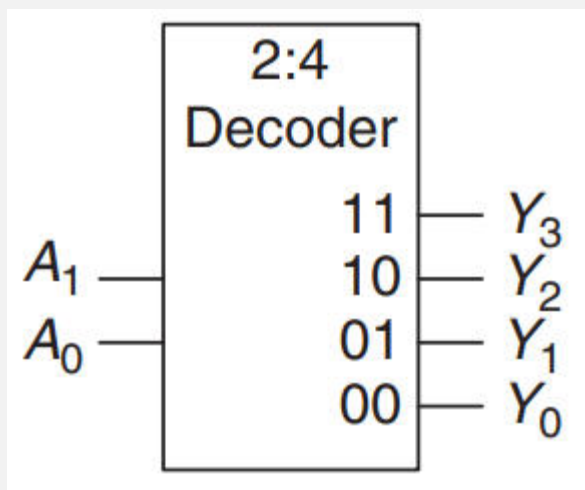
2)译码器的种类

- 普通译码器（变量译码器）
- 码制变换译码器
- 七线段显示译码器

4.6 基本组合逻辑功能部件设计

3) 普通译码器

普通译码器表示输入状态的组合逻辑，如2:4译码器



对输入的2位二进制数进行译码，具有 $2^2 = 4$ 个输出

4)2:4普通译码器设计

A_1	A_0	$\overline{Y_3}$	$\overline{Y_2}$	$\overline{Y_1}$	$\overline{Y_0}$
0	0	1	1	1	0
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	1	1	1

$$\overline{Y_3} = \overline{A_1 A_0}$$

$$\overline{Y_2} = \overline{A_1 \overline{A_0}}$$

$$\overline{Y_1} = \overline{\overline{A_1} A_0}$$

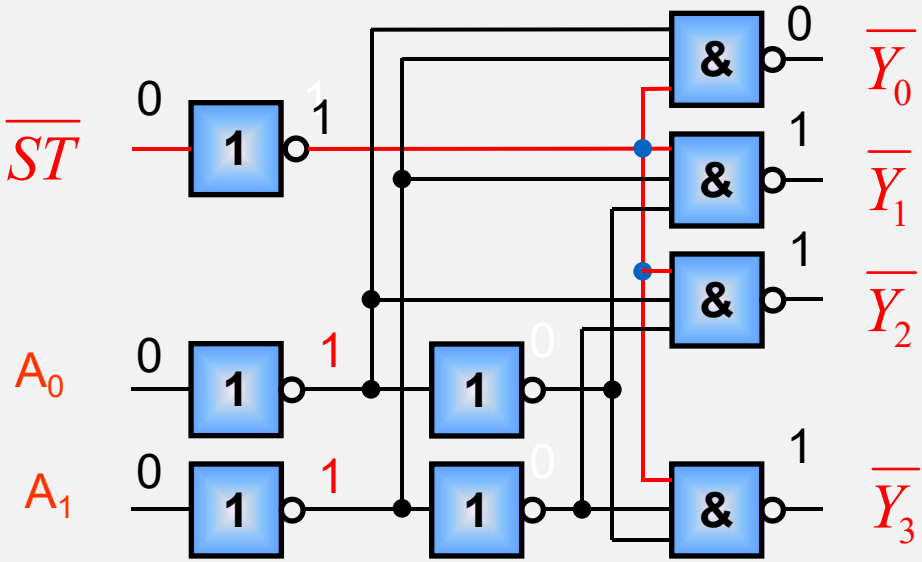
$$\overline{Y_0} = \overline{\overline{A_1} \overline{A_0}}$$

为使得1的数量变得非常少，把输出全部取非，再考察新的输出，然后再变回去。
低电平有效的符号（bar）不能参与运算。

4.6 基本组合逻辑功能部件设计

5)带选通（strobe）功能的2:4普通译码器

$\overline{ST}$	A_1	A_0	$\overline{Y_3}$	$\overline{Y_2}$	$\overline{Y_1}$	$\overline{Y_0}$
1	X	X	1	1	1	1
0	0	0	1	1	1	0
0	0	1	1	1	0	1
0	1	0	1	0	1	1
0	1	1	0	1	1	1



$$\overline{Y_3} = \overline{A_1 A_0 \overline{ST}} = \overline{A_1 A_0} ST$$

$$\overline{Y_2} = \overline{A_1 \overline{A_0} ST} \quad \overline{Y_1} = \overline{\overline{A_1} A_0 ST} \quad \overline{Y_0} = \overline{\overline{A_1} \overline{A_0} ST}$$

4.6 基本组合逻辑功能部件设计

7. 码制变换译码器

码制变换译码器：将一种码制转换为另一种码制。

例1: 设计一个将余三码（从3开始编码十进制数字0）转换为8421BCD码的转换电路

真值表（4个合并）

A	B	C	D	W	X	Y	Z
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	1
0	1	0	1	0	0	1	0
0	1	1	0	0	0	1	1
0	1	1	1	0	1	0	0
1	0	0	0	0	1	0	1
1	0	0	1	0	1	1	0
1	0	1	0	0	1	1	1
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	1

BCD码常用于传感器中定点小数的表示。

注意函数输入的定义域：[3, 12]

AB \ CD	00	01	11	10
00	X	0	1	0
01	X	0	X	0
11	0	0	X	1
10	X	0	X	0

W

$$W = AB + ACD$$

AB \ CD	00	01	11	10
00	X	0	0	1
01	X	0	X	1
11	0	1	X	0
10	X	0	X	1

X

$$X = \overline{BC} + \overline{BD} + BCD$$

不在定义域内的函数数值可以用don't care（取值任选）

4.6 基本组合逻辑功能部件设计

A	B	C	D	W	X	Y	Z
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	1
0	1	0	1	0	0	1	0
0	1	1	0	0	0	1	1
0	1	1	1	0	1	0	0
1	0	0	0	0	1	0	1
1	0	0	1	0	1	1	0
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	1

CD \ AB	AB			
	00	01	11	10
00	X	0	0	0
01	X	1	X	1
11	0	0	X	0
10	X	1	X	1

Y

$$Y = \overline{C}D + C\overline{D}$$

CD \ AB	AB			
	00	01	11	10
00	X	1	1	1
01	X	0	X	0
11	0	0	X	0
10	X	1	X	1

Z

$$Z = \overline{D}$$

4.6 基本组合逻辑功能部件设计

若只使用与非门，则

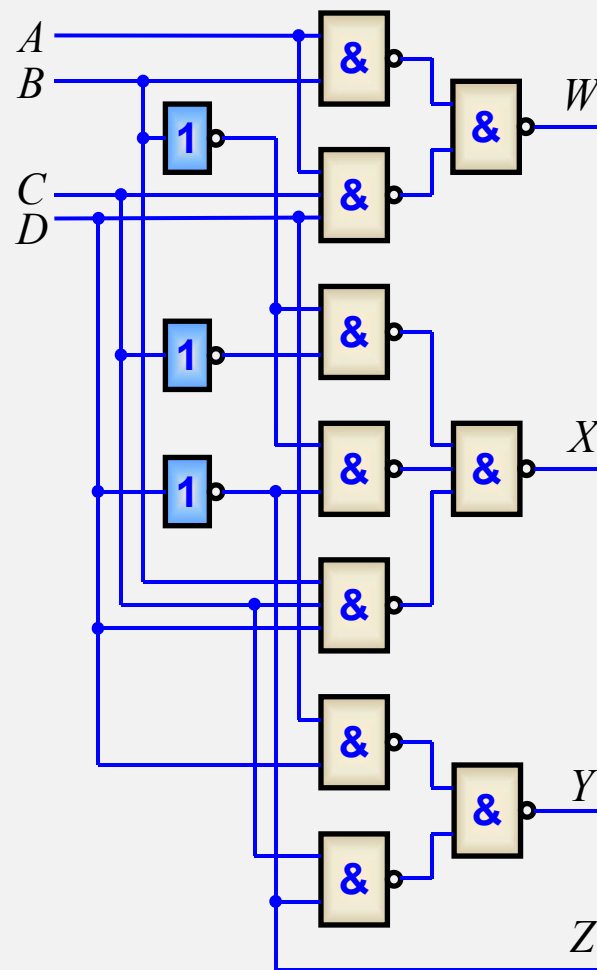
$$W = \overline{\overline{AB} \cdot \overline{ACD}}$$

$$X = \overline{\overline{AC} \cdot \overline{BD} \cdot \overline{BCD}}$$

$$Y = \overline{\overline{CD} \cdot \overline{CD}}$$

$$Z = \overline{D}$$

这个电路一共用了多少个nMOS和pMOS晶体管？

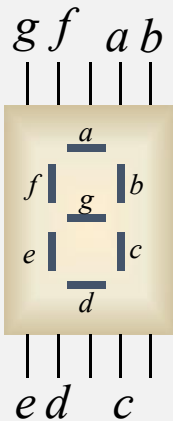
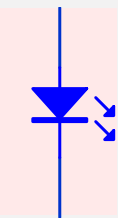


4.6 基本组合逻辑功能部件设计

8. 七线段显示译码器 (seven-segment display decoder)

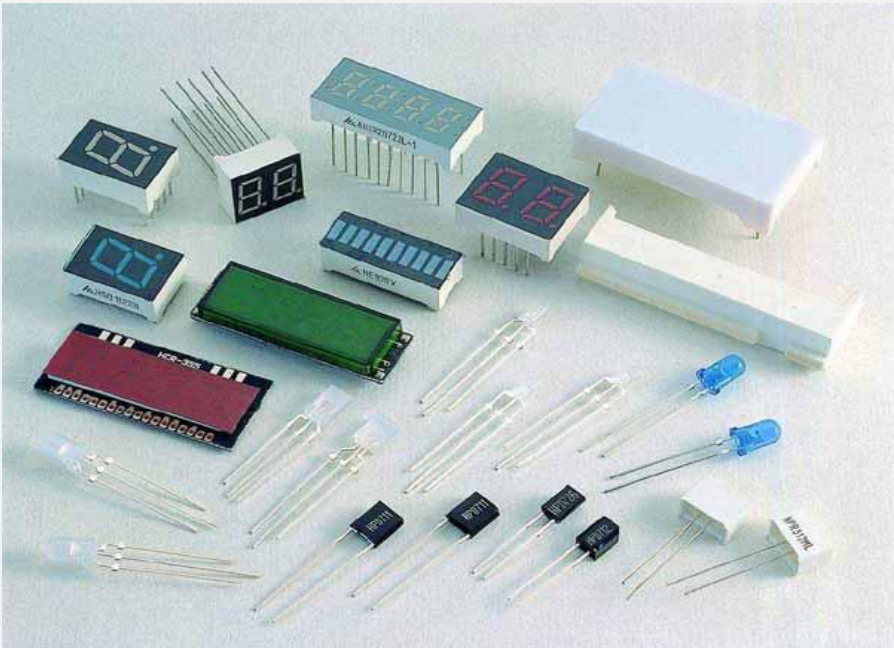
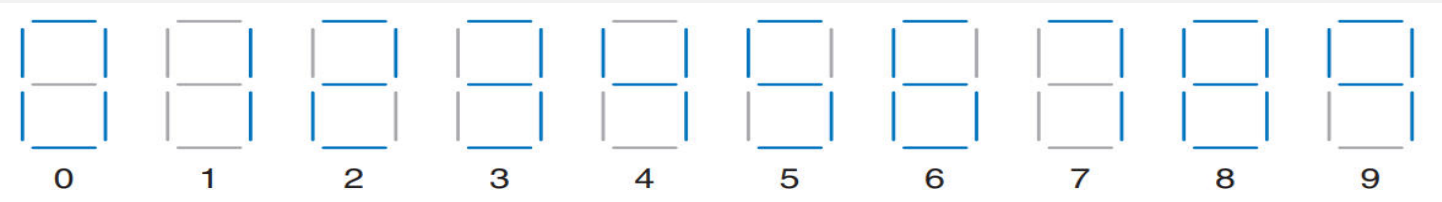
光线发射二极管，简称发光二极管。可以单独封装，也可以组合封装为数码管。

下降到模拟电路



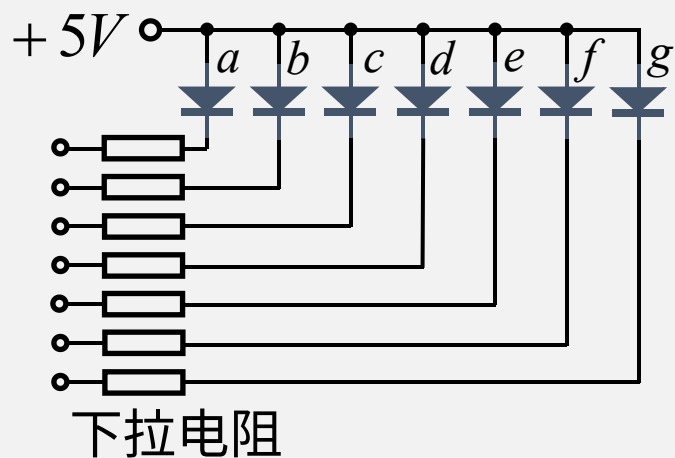
二极管 (diode) 是单向导通器件

不带小数点的显示



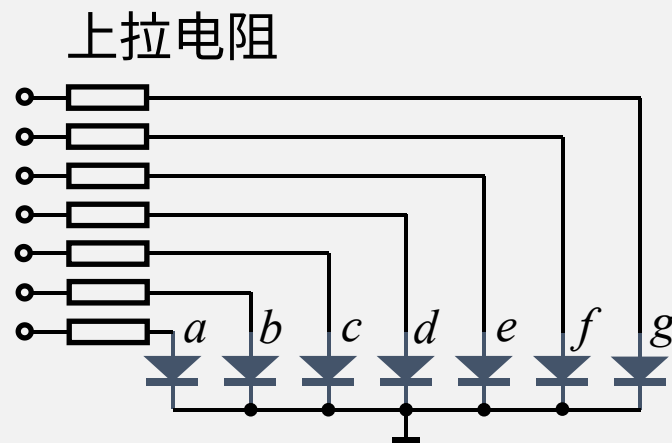
4.6 基本组合逻辑功能部件设计

发光二极管按驱动方式又分为共阳极（common anode）和共阴极（common cathode）接法。



共阳极接法

输入为低电位时，灯亮



共阴极接法

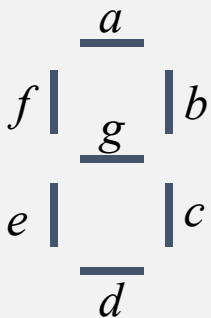
输入为高电位时，灯亮

4.6 基本组合逻辑功能部件设计

例 设计8421 BCD输入的七段（seven-segment）显示译码器。

真值表（合并版本）

D	C	B	A	a	b	c	d	e	f	g	显示
0	0	0	0	0	0	0	0	0	0	1	0
0	0	0	1	1	0	0	1	1	1	1	1
0	0	1	0	0	0	1	0	0	1	0	2
0	0	1	1	0	0	0	0	1	1	0	3
0	1	0	0	1	0	0	1	1	0	0	4
0	1	0	1	0	1	0	0	1	0	0	5
0	1	1	0	0	1	0	0	0	0	0	6
0	1	1	1	0	0	0	1	1	1	1	7
1	0	0	0	0	0	0	0	0	0	0	8
1	0	0	1	0	0	0	0	1	0	0	9



采用共阳极设计
输入为低电位时，相应的线段亮

4.6 基本组合逻辑功能部件设计

D	C	B	A	a	b	c	d	e	f	g	显示
0	0	0	0	0	0	0	0	0	0	1	0
0	0	0	1	1	0	0	1	1	1	1	1
0	0	1	0	0	0	1	0	0	1	0	2
0	0	1	1	0	0	0	0	1	1	0	3
0	1	0	0	1	0	0	1	1	0	0	4
0	1	0	1	0	1	0	0	1	0	0	5
0	1	1	0	0	1	0	0	0	0	0	6
0	1	1	1	0	0	0	1	1	1	1	7
1	0	0	0	0	0	0	0	0	0	0	8
1	0	0	1	0	0	0	0	1	0	0	9

没有落在函数定义域[0, 9]内的不合法的输入对应的输出标为don't care (值任意选)

DC \ BA					
		00	01	11	10
00	0	1	X	0	
01	1	0	X	0	
11	0	0	X	X	
10	0	0	X	X	

a

$$a = \overline{D}\overline{C}\overline{B}A + C\overline{B}\overline{A}$$

分别对（遍历）每一个输出比特，构建布尔函数，并化简。

4.6 基本组合逻辑功能部件设计

DC \ BA	00	01	11	10
00	0	0	X	0
01	0	1	X	0
11	0	0	X	X
10	0	1	X	X

b

$$b = \overline{C}BA + C\overline{B}\overline{A}$$

DC \ BA	00	01	11	10
00	0	0	X	0
01	0	0	X	0
11	0	0	X	X
10	1	0	X	X

c

$$c = \overline{C}B\overline{A}$$

DC \ BA	00	01	11	10
00	0	1	X	0
01	1	0	X	0
11	0	1	X	X
10	0	0	X	X

d

$$d = \overline{C}B\overline{A} + CBA + \overline{D}C\overline{B}A$$

DC \ BA	00	01	11	10
00	0	1	X	0
01	1	1	X	1
11	1	1	X	X
10	0	0	X	X

e

$$e = A + \overline{C}B\overline{A}$$

DC \ BA	00	01	11	10
00	0	0	X	0
01	1	0	X	0
11	1	1	X	X
10	1	0	X	X

f

$$f = BA + \overline{C}B\overline{A} + \overline{D}C\overline{B}A$$

DC \ BA	00	01	11	10
00	1	0	X	0
01	1	0	X	0
11	0	1	X	X
10	0	0	X	X

g

$$g = \overline{D}C\overline{B} + CBA$$

4.6 基本组合逻辑功能部件设计

$$a = \overline{D}\overline{C}\overline{B}A + C\overline{B}\overline{A}$$

$$b = C\overline{B}A + CBA\overline{A}$$

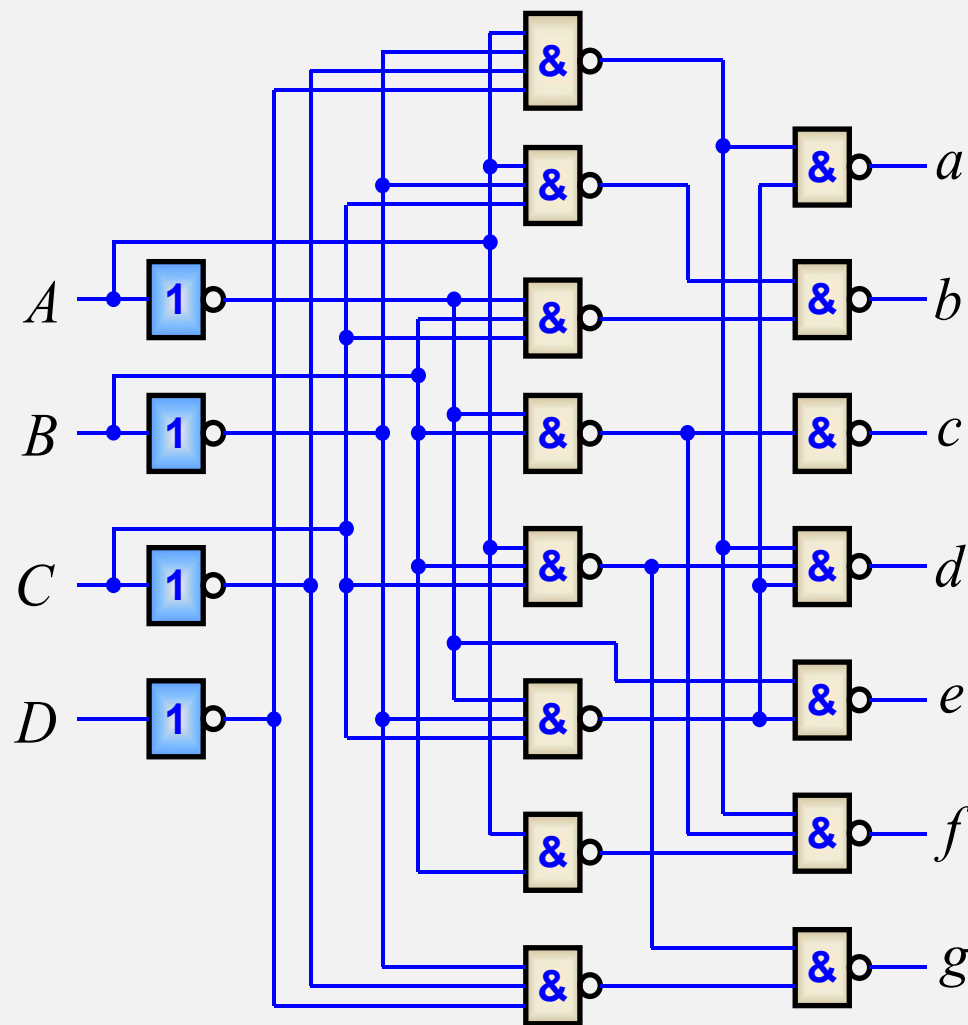
$$c = \overline{C}B\overline{A}$$

$$d = C\overline{B}\overline{A} + CBA + \overline{D}C\overline{B}A$$

$$e = A + C\overline{B}\overline{A}$$

$$f = BA + \overline{C}B\overline{A} + \overline{D}C\overline{B}A$$

$$g = \overline{D}\overline{C}\overline{B} + CBA$$

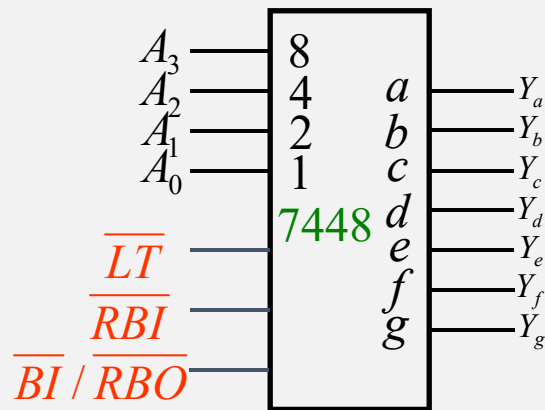


对于有多个输出的组合逻辑电路，提取公因式
(蕴含项及其组成部分)，对相同的电路进行共用。

化为用NAND门实现

4.6 基本组合逻辑功能部件设计

动态显示



灯测试输入端主要用于检查LED的好坏。

$\overline{LT} = \begin{cases} 1 & \text{时, 正常译码。} \\ 0 & \text{时, 输出 } a \sim g \text{ 全1, 七段全亮。} \end{cases}$

消隐输入端（与灭0输出端共用）

$\overline{BI} = \begin{cases} 1 & \text{时, 正常译码。} \\ 0 & \text{时, 不管输入何种状态, 输出全0} \end{cases}$

灭0输入端，熄灭无意义的0

$\overline{RBI} = \begin{cases} 0 & \text{时, 灭掉不要显示的0, } 001 \rightarrow 1 \\ 1 & \text{时, 显示0, 不灭中间0。 } 101 \rightarrow 101 \end{cases}$

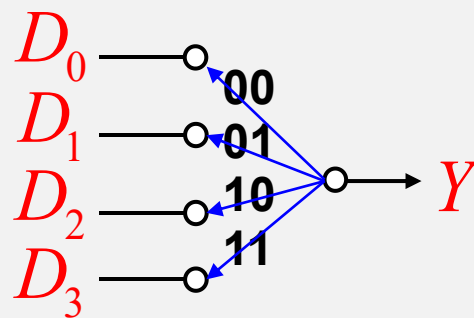
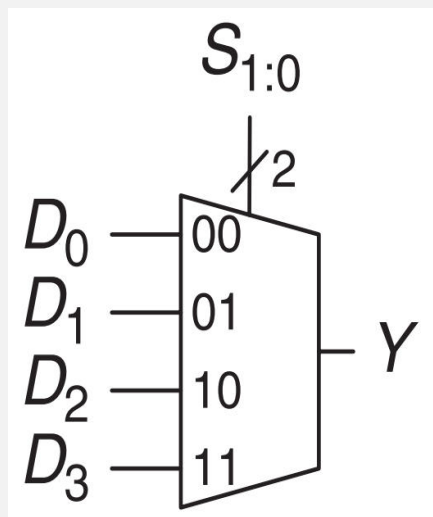
灭0输出端 $\overline{RBO}$ 与灭0输入端配合使用

即：灭0输入等于0,灭0输出一定等于0。既可作为输入，也可作为输出

4.7 多路复用器 (multiplexer, MUX)

多路复用器的基本功能

多路复用器：从一组1-bit输入数据中，选择**某一个**1-bit数据作为输出，也称为**多路选择器**。



单刀多掷开关模型

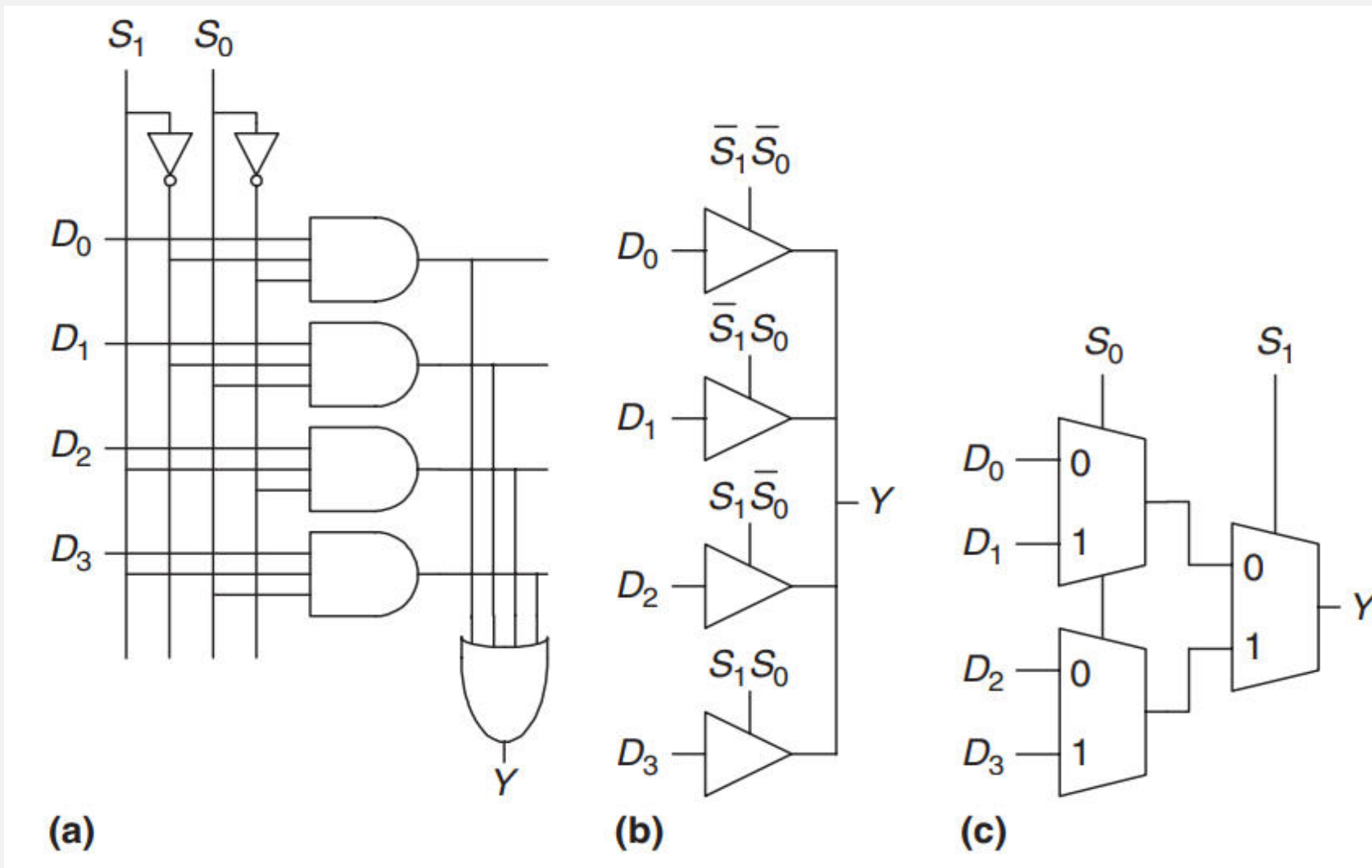
D	A ₁	A ₀	Y
D ₀	0	0	D ₀
D ₁	0	1	D ₁
D ₂	1	0	D ₂
D ₃	1	1	D ₃



$$Y = \overline{A_1} \overline{A_0} D_0 + \overline{A_1} A_0 D_1 + A_1 \overline{A_0} D_2 + A_1 A_0 D_3$$

4.7 多路复用器(multiplexer)

4路复用器的三种不同设计



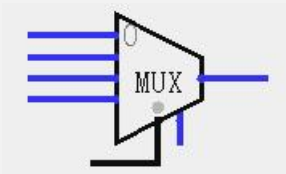
相当于C高级
编程语言里
switch case
语句的功能

4.7 多路复用器(multiplexer)

带选通 (strobe) 信号的多路复用器

$\overline{ST_1}$	A_1	A_0	Y_1
1	X	X	0
0	0	0	D_0
0	0	1	D_1
0	1	0	D_2
0	1	1	D_3

$$Y = \overline{\overline{ST_1}}(\overline{\overline{A_1}}\overline{\overline{A_0}}D_0 + \overline{\overline{A_1}}A_0D_1 + A_1\overline{\overline{A_0}}D_2 + A_1A_0D_3)$$



低电平有效的bar符号应与其下面的字母看作一个整体信号。
该bar符号的意义不同，不能参与布尔运算。

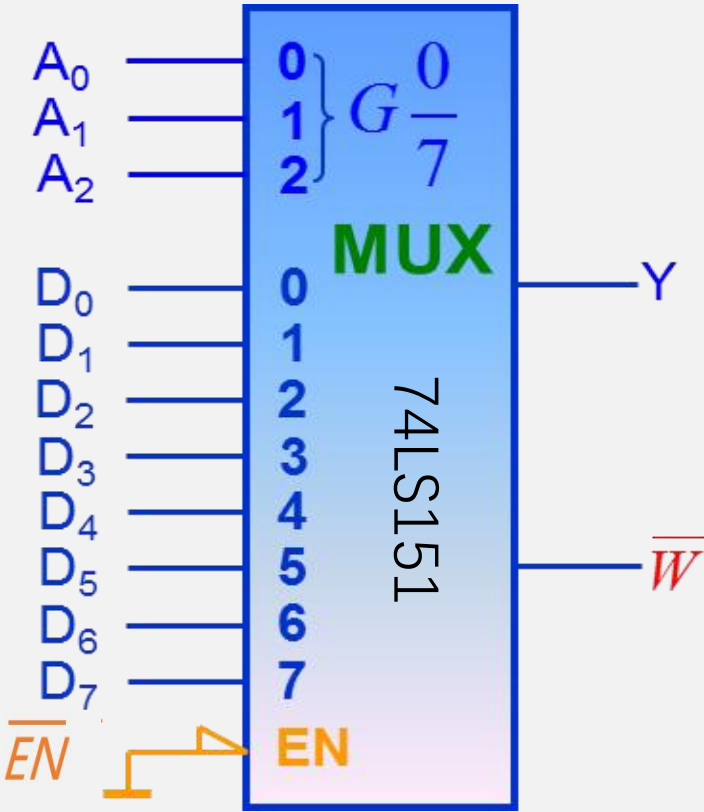
4.7 多路复用器(multiplexer)

带使能的多路复用器设计

$\overline{EN}$	A_2	A_1	A_0	Y	$\overline{W}$
1	X	X	X	0	1
0	0	0	0	D_0	$\overline{D_0}$
0	0	0	1	D_1	$\overline{D_1}$
0	0	1	0	D_2	$\overline{D_2}$
0	0	1	1	D_3	$\overline{D_3}$
0	1	0	0	D_4	$\overline{D_4}$
0	1	1	1	D_5	$\overline{D_5}$
0	1	0	0	D_6	$\overline{D_6}$
0	1	1	1	D_7	$\overline{D_7}$

$\overline{EN}$: 使能信号, 低有效。

$Y, \overline{W}$: 互补输出端。



$$Y = (\overline{A_2}\overline{A_1}\overline{A_0} D_0 + \overline{A_2}\overline{A_1}A_0 D_1 + \overline{A_2}A_1\overline{A_0} D_2 + \overline{A_2}A_1A_0 D_3 + A_2\overline{A_1}\overline{A_0} D_4 + A_2\overline{A_1}A_0 D_5 + A_2A_1\overline{A_0} D_6 + A_2A_1A_0 D_7) \overline{EN}$$

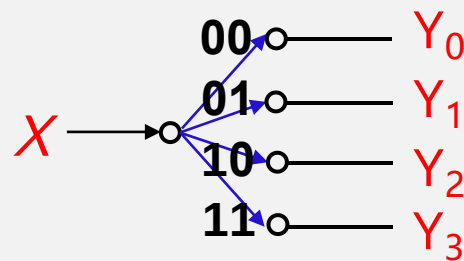
多路复用器在微架构中的应用



4.8 多路分配器(解复用器, demultiplexer)

基本功能

将1个输入数据，根据需要传送到m个输出端中的一个指定输出端的电路，称为多路分配器或解复用器。

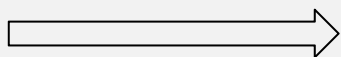


demultiplexer, DEMUX

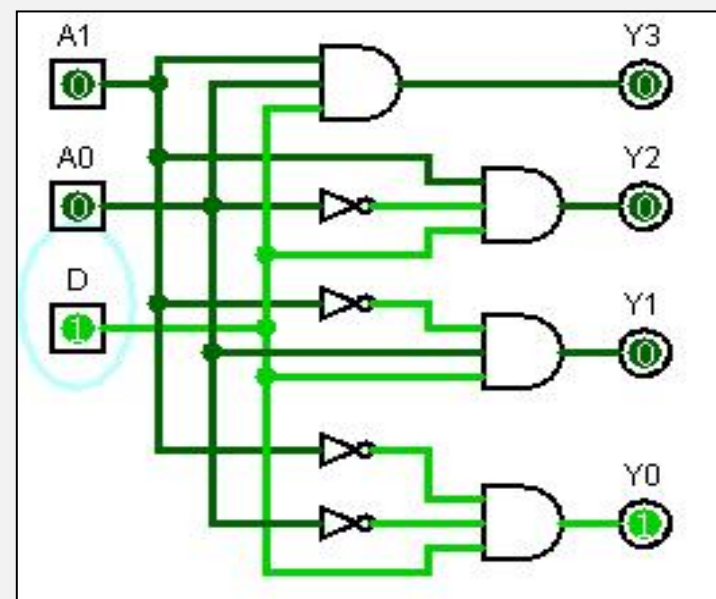
4.8 多路分配器(解复用器, demultiplexer)

多路分配器的设计

A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	D
0	1	0	0	D	0
1	0	0	D	0	0
1	1	D	0	0	0



$$\begin{aligned} Y_0 &= \overline{A_1} \overline{A_0} D & Y_1 &= \overline{A_1} A_0 D \\ Y_2 &= A_1 \overline{A_0} D & Y_3 &= A_1 A_0 D \end{aligned}$$

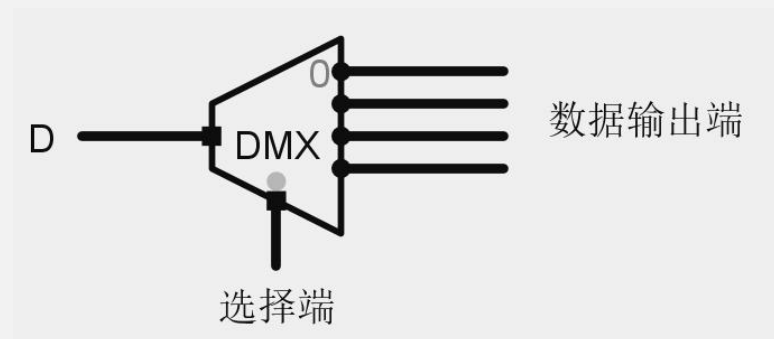


4.9 对比：译码器、多路分配器、多路复用器



$$Y_0 = \overline{A_1}\overline{A_0} \quad Y_1 = \overline{A_1}A_0 \quad Y_2 = A_1\overline{A_0} \quad Y_3 = A_1A_0$$

译码器：选中的行，输出一定为高



$$Y_0 = \overline{A_1}\overline{A_0}D \quad Y_1 = \overline{A_1}A_0D \quad Y_2 = A_1\overline{A_0}D \quad Y_3 = A_1A_0D$$

多路分配器：选中的行，输出取决于输入



$$Y = \overline{A_1}\overline{A_0}D_0 + \overline{A_1}A_0D_1 + A_1\overline{A_0}D_2 + A_1A_0D_3$$

多路复用器：多路分配器的逆过程，选择一路进行连通

4.10 基于译码器的组合逻辑

译码器逻辑 (decoder logic)：利用译码器实现组合逻辑功能
类似地，还有**多路复用器逻辑 (multiplexer Logic)**

一个n输入的译码器，其输出包含了n个输入变量的**全部最小项**。用n输入译码器加**逻辑门**就能实现任何形式的输入变量个数不大于n 的组合逻辑。

A_1	A_0	Y3	Y2	Y1	Y0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

$$\begin{aligned} Y_3 &= A_1 A_0 & Y_2 &= A_1 \bar{A}_0 \\ Y_1 &= \bar{A}_1 A_0 & Y_0 &= \bar{A}_1 \bar{A}_0 \end{aligned}$$

例1 用译码器实现一组三输入变量的多输出函数，用3-8译码器实现

$$\begin{cases} F_1 = \overline{A}\overline{B} + \overline{B}C + AC \\ F_2 = \overline{A}\overline{B} + \overline{B}\overline{C} + ABC \\ F_3 = \overline{A}C + BC + A\overline{C} \end{cases}$$

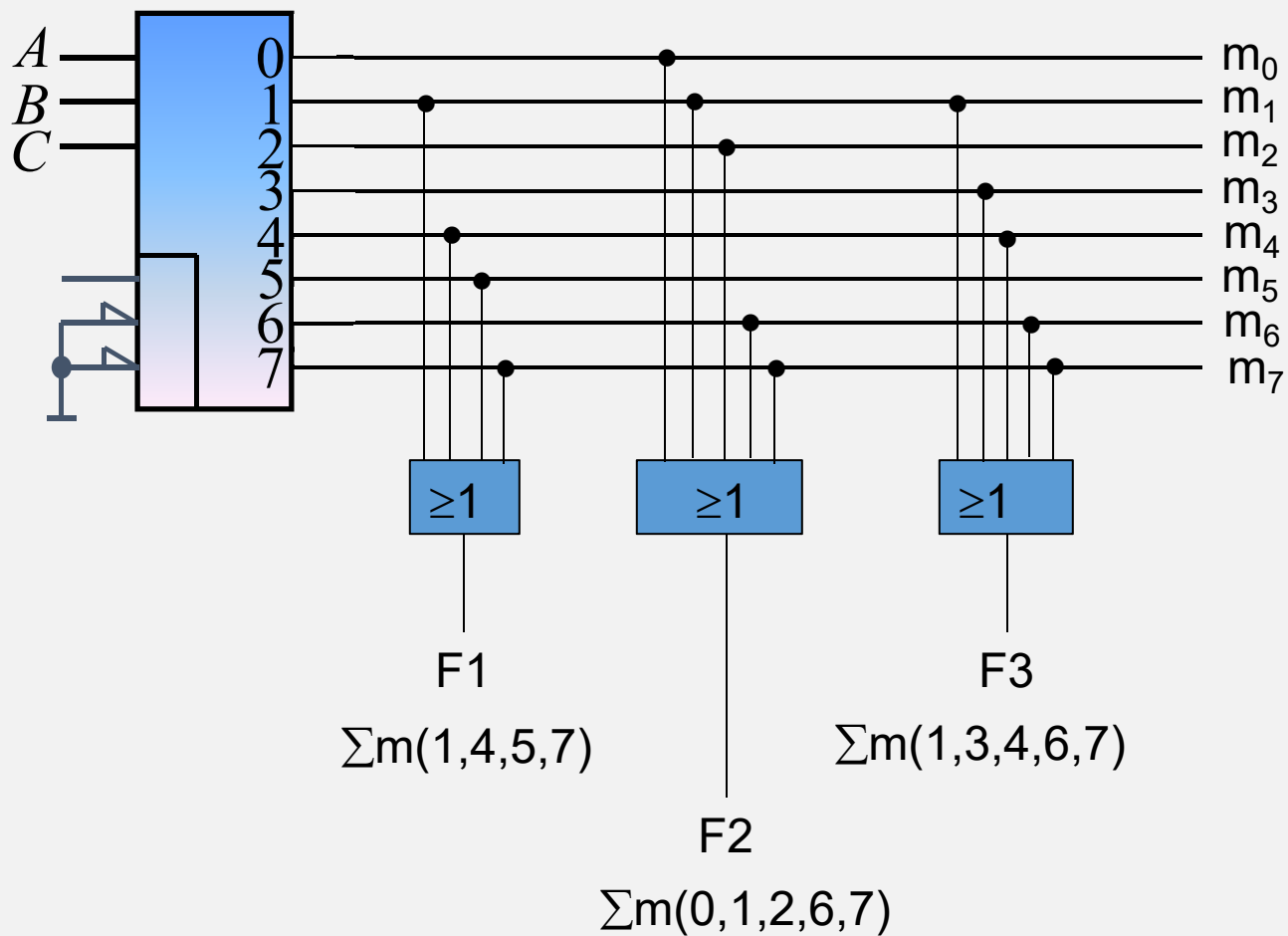
将多输出函数写成最小项的与,再搭配适当的逻辑门。

$$F_1 = \overline{A}\overline{B} + \overline{B}C + AC = \sum m(1,4,5,7)$$

$$F_2 = \overline{A}\overline{B} + \overline{B}\overline{C} + ABC = \sum m(0,1,2,6,7)$$

$$F_3 = \overline{A}C + BC + A\overline{C} = \sum m(1,3,4,6,7)$$

4.10 基于译码器的组合逻辑



4.10 基于译码器的组合逻辑

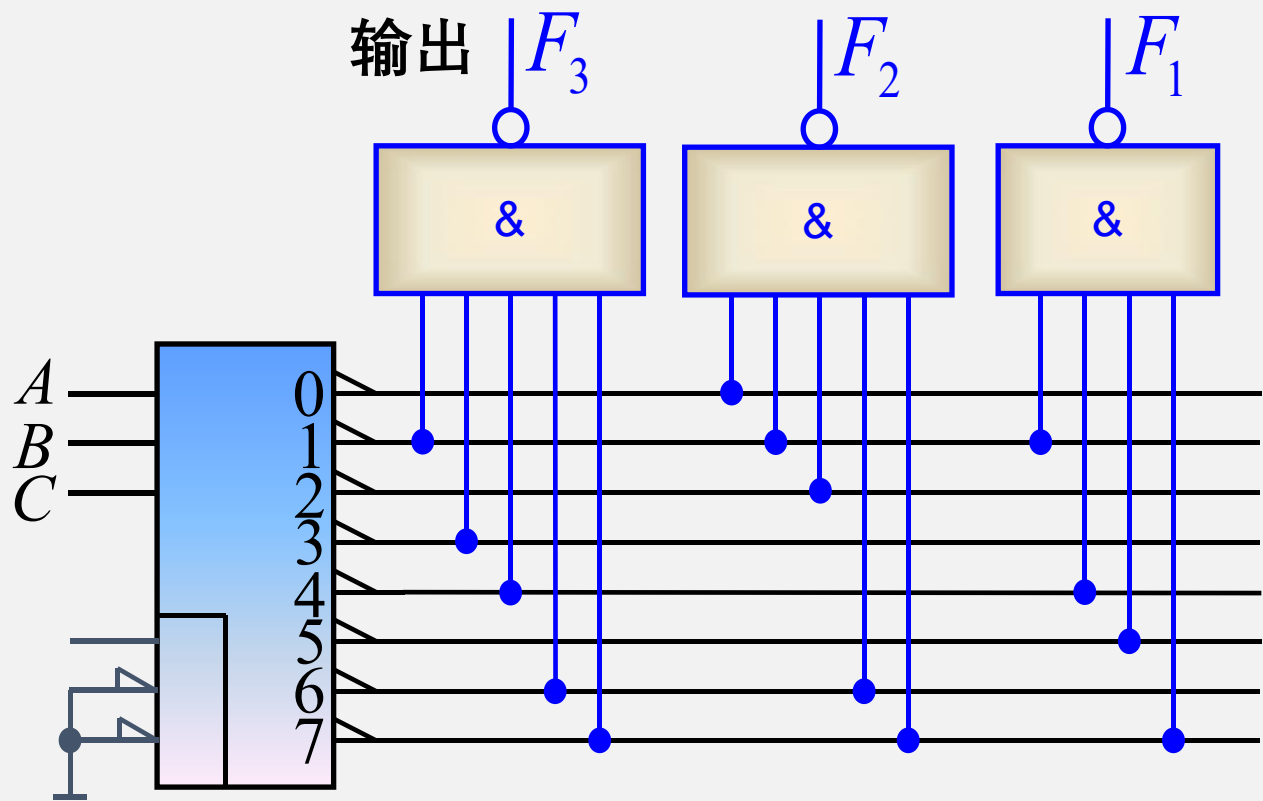
若译码器是以补变量形式输出，即输出的是 $\overline{m}_i$ ，则利用德摩根定理：

$$\begin{aligned} F_1 &= \overline{A}\overline{B} + \overline{B}C + AC = m_1 + m_4 + m_5 + m_7 \\ &= \overline{\overline{m_1 + m_4 + m_5 + m_7}} = \overline{\overline{m_1} \cdot \overline{m_4} \cdot \overline{m_5} \cdot \overline{m_7}} \\ &= \overline{\overline{Y_1} \cdot \overline{Y_4} \cdot \overline{Y_5} \cdot \overline{Y_7}} \end{aligned}$$

$$\begin{aligned} F_2 &= \overline{A}\overline{B} + \overline{B}C + ABC = \sum m(0,1,2,6,7) = \overline{\overline{m_0} \cdot \overline{m_1} \cdot \overline{m_2} \cdot \overline{m_6} \cdot \overline{m_7}} \\ &= \overline{\overline{Y_0} \cdot \overline{Y_1} \cdot \overline{Y_2} \cdot \overline{Y_6} \cdot \overline{Y_7}} \end{aligned}$$

$$F_3 = \overline{A}C + BC + A\overline{C} = \sum m(1,3,4,6,7) = \overline{\overline{Y_1} \cdot \overline{Y_3} \cdot \overline{Y_4} \cdot \overline{Y_6} \cdot \overline{Y_7}}$$

4.10 基于译码器的组合逻辑



4.10 基于译码器的组合逻辑

例2：用2个2输入译码器和适当的逻辑门实现逻辑函数组：

$$\begin{cases} F_1 = \overline{A}\overline{B} + \overline{B}C + AC \\ F_2 = \overline{A}\overline{B} + B\overline{C} + ABC \\ F_3 = \overline{A}C + BC + A\overline{C} \end{cases}$$

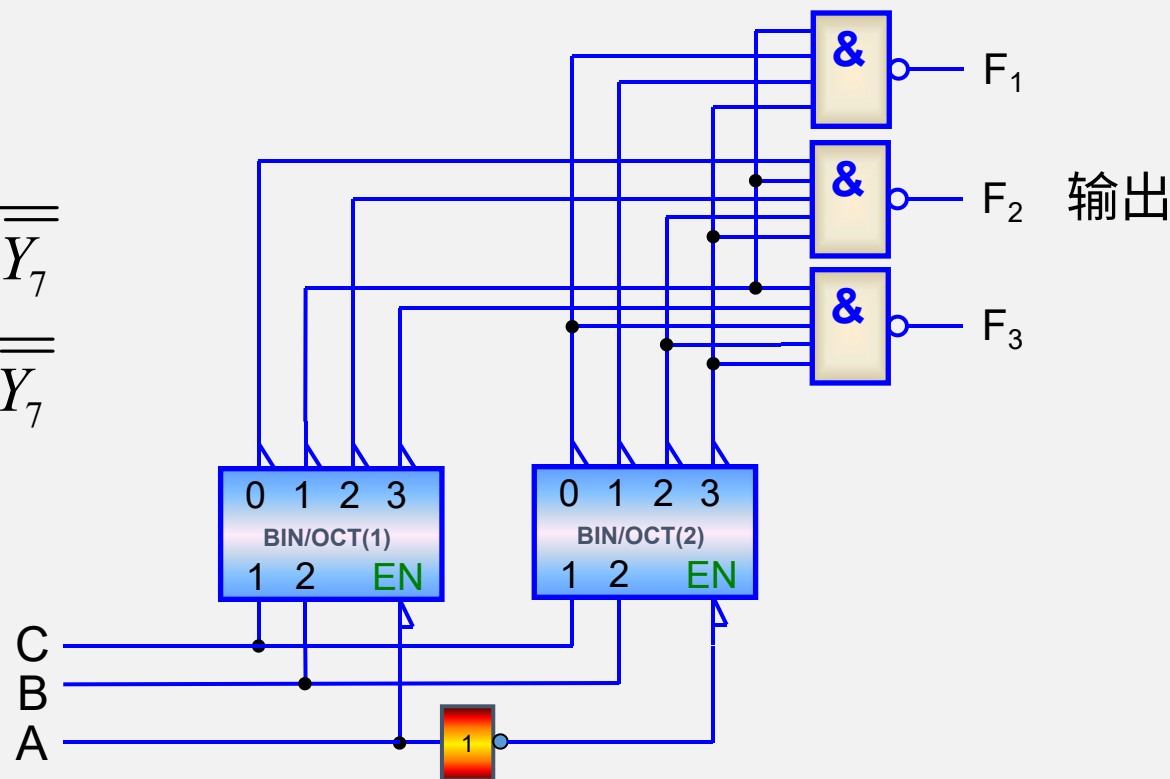
$$F_1 = \overline{A}\overline{B} + \overline{B}C + AC = \overline{m_1 \cdot m_4 \cdot m_5 \cdot m_7} = \overline{Y_1 \cdot Y_4 \cdot Y_5 \cdot Y_7}$$

$$F_2 = \overline{A}\overline{B} + B\overline{C} + ABC = \overline{m_0 \cdot m_1 \cdot m_2 \cdot m_6 \cdot m_7} = \overline{Y_0 \cdot Y_1 \cdot Y_2 \cdot Y_6 \cdot Y_7}$$

$$F_3 = \overline{A}C + BC + A\overline{C} = \overline{m_1 \cdot m_3 \cdot m_4 \cdot m_6 \cdot m_7} = \overline{Y_1 \cdot Y_3 \cdot Y_4 \cdot Y_6 \cdot Y_7}$$

4.10 基于译码器的组合逻辑

$$F_1 = \overline{Y_1} \cdot \overline{Y_4} \cdot \overline{Y_5} \cdot \overline{Y_7}$$
$$F_2 = \overline{Y_0} \cdot \overline{Y_1} \cdot \overline{Y_2} \cdot \overline{Y_6} \cdot \overline{Y_7}$$
$$F_3 = \overline{Y_1} \cdot \overline{Y_3} \cdot \overline{Y_4} \cdot \overline{Y_6} \cdot \overline{Y_7}$$



A作为最高位，作芯片选择（chip select），完成了2个译码器的横向扩展。